

Relatório

Programação Assistida e Automação com IA

Identificação

1. Problema
2. Entrada
3. Processamento
4. Saída esperada
5. Prompt utilizado
6. Código inicial
7. Análise crítica
8. Casos de teste
9. Problemas encontrados
10. Prompt de refatoração
11. Código refatorado
12. Comparação
13. Reflexão
14. Take Away

Anexo A — Autoavaliação segundo os critérios da atividade

Anexo B — Checklist de entrega

Referências

Programação Assistida e Automação com IA

Desafio Hands On — Programação Assistida e Automação com IA UDF Centro Universitário | Tendências em Ciência da Computação | Unidade II | Profa. Kadidja Valéria

Identificação

- **Nome:** Melissa Aragão Leite
- **Turma:** Tendências em Ciência da Computação — Unidade II (Programação Assistida)
- **Data:** 11/09/2026
- **Ferramenta de IA utilizada:** Claude (Anthropic)
- **Problema escolhido:** Opção E — Problema proposto pela estudante (Jogo da Forca)

1. Problema

Construir um Jogo da Forca completo: sortear uma palavra, permitir que o jogador tente adivinhá-la letra por letra, controlar o número de erros permitidos e informar vitória ou derrota ao final. Optei por um problema proposto por mim (Opção E) em vez das opções A-D porque ele tem entrada, processamento e saída tão claramente definíveis quanto um script de automação, e permite aplicar o mesmo ciclo de geração → teste → refatoração pedido nesta atividade.

2. Entrada

Uma letra por vez, digitada pelo jogador via teclado, durante toda a partida (várias entradas ao longo da execução, não um valor único no início).

3. Processamento

- sortear uma palavra (e, na versão refatorada, também uma categoria) de um banco de palavras;
- validar se a entrada do jogador é uma única letra do alfabeto;
- verificar se a letra pertence à palavra sorteada;
- atualizar o estado do jogo (letras já tentadas, número de erros);
- verificar, a cada rodada, se o jogador venceu (todas as letras descobertas) ou perdeu (erros atingiram o limite).

4. Saída esperada

A cada rodada: a palavra com as letras já descobertas e `_` nas ainda ocultas, o desenho ASCII da forca correspondente ao número de erros, o número de tentativas restantes e as letras erradas já tentadas. Ao final: mensagem de vitória (revelando a palavra) ou de derrota.

5. Prompt utilizado

Prompt de contexto, entregue à IA junto com a versão inicial do código (seção 6), seguindo a estrutura PAPEL + CONTEXTO + TAREFA sugerida no material:

Atue como um dev Python Sênior. Este é um jogo da forca funcional, mas escrito como uma primeira versão rápida, tudo em um arquivo. Avalie os problemas de design antes de sugerir qualquer mudança: separação de responsabilidades, validação de entrada, estrutura de dados usada para o estado do jogo.

6. Código inicial

```

# Jogo da Forca - Versao 1 (ANTES)
# Primeira versao escrita manualmente, sem apoio de IA.
# Objetivo: apenas fazer o jogo funcionar (logica central).

palavras = ["python", "computador", "faculdade", "programacao", "internet"]

import random

palavra = random.choice(palavras)
letras_certas = []
letras_erradas = []
tentativas = 6

print("Bem vindo ao jogo da forca!")

fim = False
while fim == False:
    print("")
    palavra_mostrada = ""
    i = 0
    while i < len(palavra):
        letra = palavra[i]
        if letra in letras_certas:
            palavra_mostrada = palavra_mostrada + letra
        else:
            palavra_mostrada = palavra_mostrada + "_"
        i = i + 1
    print(palavra_mostrada)
    print("Tentativas restantes: " + str(tentativas))
    print("Letras erradas: " + str(letras_erradas))

    chute = input("Digite uma letra: ")

    if chute in palavra:
        if chute not in letras_certas:
            letras_certas.append(chute)
    else:
        if chute not in letras_erradas:
            letras_erradas.append(chute)
            tentativas = tentativas - 1

    ganhou = True
    for letra in palavra:
        if letra not in letras_certas:
            ganhou = False

    if ganhou == True:
        print("")
        print("Parabens, voce venceu! A palavra era: " + palavra)
        fim = True

    if tentativas == 0:
        print("")
        print("Voce perdeu! A palavra era: " + palavra)
        fim = True

```

7. Análise crítica

A primeira versão funciona, mas apresenta problemas típicos de um primeiro rascunho:

1. Tudo em um único arquivo/escopo, sem funções — lógica do jogo, exibição e laço principal estão misturados.
2. Quatro variáveis soltas (`palavra`, `letras_certas`, `letras_erradas`, `tentativas`) representando o estado do jogo, sem nenhuma estrutura que as agrupe.
3. Nenhuma validação de entrada: digitar `" "`, `"ab"` ou um número não gera erro controlado, apenas um comportamento estranho na comparação `chute in palavra`.
4. Não impede repetir uma letra já tentada — o jogador podia perder uma tentativa de novo numa letra errada repetida.
5. `letras_certas.append(...)` com checagem manual de duplicidade — uma lista fazendo o papel de um `set`, mais lento e mais verboso.
6. Verificação de vitória feita com `for` + variável booleana recalculada a cada rodada, em vez de uma

expressão direta.

7. Nenhum teste automatizado — a única forma de saber se o jogo funcionava era jogar manualmente.

8. Casos de teste

Teste 1 — Caso normal

Entrada válida
→ resultado esperado

Palavra “sol”, jogador digita `s`, `o`, `l` em sequência → jogo declara vitória e revela a palavra completa.

Resultado: conforme esperado
(`tests/test_forca.py::TestCondicoesDeFimDeJogo::test_venceu_quando_todas_as_letras_foram_tentadas`).

Teste 2 — Caso limite

Entrada vazia, lista pequena ou valor extremo
→ comportamento esperado

Jogador digita `""` (Enter sem nada) ou uma letra repetida. **Resultado esperado:** a entrada é rejeitada com mensagem clara, sem consumir uma tentativa. **Resultado obtido na v1:** falhava — entrada vazia não era tratada e letra repetida descontava tentativa. **Resultado obtido na v2 (depois da refatoração):** conforme esperado (`TestNormalizarLetra::test_rejeita_string_vazia`, `TestRegistrarTentativa::test_rejeita_letra_repetida`).

Teste 3 — Caso de erro

Entrada inválida
→ tratamento esperado

Jogador digita `"5"`, `"ab"` ou `"-"` (um caractere que não é letra do alfabeto). **Resultado esperado:** mensagem de erro pedindo uma letra válida, sem travar o jogo. **Resultado obtido:** funcionou para a maioria dos casos, mas revelou um bug real descrito na seção 9.

9. Problemas encontrados

Além dos problemas de design listados na seção 7, um teste manual jogando a versão refatorada no terminal (fora dos testes automatizados) revelou um bug real: ao sortear a palavra “**guarda-chuva**”, o jogo travava pedindo letra indefinidamente, mesmo com todas as letras corretas já digitadas.

Causa raiz: a palavra tem um hífen. A validação de entrada rejeita corretamente qualquer caractere que não seja uma letra do alfabeto — então o hífen nunca poderia ser “adivinhado” como palpite, e a condição de vitória original exigia que *todos* os caracteres da palavra, hífen incluído, estivessem entre as letras tentadas. Resultado: uma palavra estruturalmente impossível de vencer.

Esse achado é um bom exemplo de por que a Etapa 4 (testar a solução) não pode ser pulada mesmo quando os testes automatizados já passam: os testes cobriram a lógica pensada de antemão, mas foi jogando de verdade que o caso de borda apareceu.

10. Prompt de refatoração

Prompts estruturados usados nas diferentes rodadas de refatoração:

Refatore a lógica de estado do jogo (palavra, letras certas/erradas, tentativas) separando a verificação de letras em uma função específica. Evite mutação de estado desnecessária e comente as alterações. Não misture lógica do jogo com print/input — separe em módulos.

As letras já tentadas estão em duas listas com checagem manual de duplicidade. Isso pode ser modelado com um set? Justifique a mudança em termos de legibilidade e desempenho.

Prompt de depuração usado especificamente para o bug da seção 9:

O jogo trava pedindo letra pra sempre quando a palavra sorteada é 'guarda-chuva', mesmo com todas as letras certas já digitadas. A entrada é validada com `letra.isalpha()`, então o hífen nunca é aceito como palpite. Como resolver sem remover palavras compostas do banco?

11. Código refatorado

O código foi dividido em quatro módulos (`palavras.py`, `desenho.py`, `forca.py`, `main.py`), listados a seguir.

palavras.py — banco de palavras e sorteio:

```
"""Banco de palavras do jogo da forca."""

from dataclasses import dataclass
import random

BANCO_DE_PALAVRAS: dict[str, list[str]] = {
    "tecnologia": ["python", "algoritmo", "computador", "internet", "software"],
    "faculdade": ["faculdade", "programacao", "professor", "biblioteca", "monitoria"],
    "cotidiano": ["cachorro", "guarda-chuva", "bicicleta", "chocolate", "aeroporto"],
}

@dataclass(frozen=True)
class Desafio:
    palavra: str
    categoria: str

def escolher_palavra(categoria: str | None = None) -> Desafio:
    if categoria is not None:
        if categoria not in BANCO_DE_PALAVRAS:
            raise ValueError(f"Categoria desconhecida: {categoria!r}")
        categorias_candidatas = [categoria]
    else:
        categorias_candidatas = list(BANCO_DE_PALAVRAS.keys())

    categoria_sorteada = random.choice(categorias_candidatas)
    palavra_sorteada = random.choice(BANCO_DE_PALAVRAS[categoria_sorteada])
    return Desafio(palavra=palavra_sorteada, categoria=categoria_sorteada)
```

desenho.py — arte ASCII da forca:

```
"""Desenho ASCII da forca."""

ESTAGIOS_DA_FORCA: list[str] = [
    # ... 7 estagios, do vazio ao boneco completo (ver arquivo no repositorio) ...
]

MAX_ERROS: int = len(ESTAGIOS_DA_FORCA) - 1

def desenhar_forca(numero_de_erros: int) -> str:
    indice = max(0, min(numero_de_erros, MAX_ERROS))
    return ESTAGIOS_DA_FORCA[indice]
```

forca.py — lógica pura do jogo (o núcleo testável):

```

"""Logica central (pura) do jogo da forca."""

from dataclasses import dataclass, field

@dataclass
class EstadoDoJogo:
    palavra: str
    letras_tentadas: set[str] = field(default_factory=set)
    max_erros: int = 6

    @property
    def letras_erradas(self) -> set[str]:
        return {letra for letra in self.letras_tentadas if letra not in self.palavra}

    @property
    def numero_de_erros(self) -> int:
        return len(self.letras_erradas)

    @property
    def tentativas_restantes(self) -> int:
        return self.max_erros - self.numero_de_erros

def normalizar_letra(entrada: str) -> str:
    letra = entrada.strip().lower()
    if len(letra) != 1 or not letra.isalpha():
        raise ValueError("Digite exatamente uma letra (a-z).")
    return letra

def registrar_tentativa(estados: EstadoDoJogo, letra: str) -> EstadoDoJogo:
    if letra in estados.letras_tentadas:
        raise ValueError(f"A letra {letra!r} ja foi tentada.")
    novas_letras = estados.letras_tentadas | {letra}
    return EstadoDoJogo(
        palavra=estados.palavra,
        letras_tentadas=novas_letras,
        max_erros=estados.max_erros,
    )

def _precisa_ser_adivinhada(caractere: str) -> bool:
    # Correcao do bug da secao 9: hifen, espaco etc. nao precisam ser
    # "adivinhados" como uma letra, senao a palavra fica invenciivel.
    return caractere.isalpha()

def palavra_revelada(estados: EstadoDoJogo) -> str:
    return " ".join(
        letra
        if not _precisa_ser_adivinhada(letra) or letra in estados.letras_tentadas
        else "_"
        for letra in estados.palavra
    )

def venceu(estados: EstadoDoJogo) -> bool:
    return all(
        letra in estados.letras_tentadas
        for letra in estados.palavra
        if _precisa_ser_adivinhada(letra)
    )

def perdeu(estados: EstadoDoJogo) -> bool:
    return estados.numero_de_erros >= estados.max_erros

def jogo_terminado(estados: EstadoDoJogo) -> bool:
    return venceu(estados) or perdeu(estados)

```

main.py — entrada/saída (I/O) e laço do jogo:

```

"""Ponto de entrada do jogo da forca (versao 2 - DEPOIS)."""

from desenho import desenhar_forca
from forca import (
    EstadoDoJogo,
    jogo_terminado,
    normalizar_letra,
    palavra_revelada,
    registrar_tentativa,
    venceu,
)
from palavras import escolher_palavra

def imprimir_estado(estado: EstadoDoJogo) -> None:
    print(desenhar_forca(estado.numero_de_erros))
    print("Palavra:", palavra_revelada(estado))
    print("Tentativas restantes:", estado.tentativas_restantes)
    if estado.lettras_erradas:
        letras = ", ".join(sorted(estado.lettras_erradas))
        print("Letras erradas:", letras)
    print()

def ler_letra_do_jogador(estado: EstadoDoJogo) -> str:
    while True:
        entrada = input("Digite uma letra: ")
        try:
            letra = normalizar_letra(entrada)
        except ValueError as erro:
            print(f"Entrada invalida: {erro}")
            continue

        if letra in estado.lettras_tentadas:
            print("Voce ja tentou essa letra, escolha outra.")
            continue

        return letra

def jogar_uma_partida(categoria: str | None = None) -> None:
    desafio = escolher_palavra(categoria)
    estado = EstadoDoJogo(palavra=desafio.palavra)

    print("=" * 40)
    print("JOGO DA FORCA")
    print(f"Categoria sorteada: {desafio.categoria}")
    print("=" * 40)

    while not jogo_terminado(estado):
        imprimir_estado(estado)
        letra = ler_letra_do_jogador(estado)
        estado = registrar_tentativa(estado, letra)

    imprimir_estado(estado)
    if venceu(estado):
        print(f"Parabens! Voce venceu! A palavra era '{estado.palavra}'.")
    else:
        print(f"Voce perdeu! A palavra era '{estado.palavra}'.")

def main() -> None:
    continuar = True
    while continuar:
        jogar_uma_partida()
        resposta = input("\nJogar novamente? (s/n): ").strip().lower()
        continuar = resposta.startswith("s")
    print("Ate a proxima!")

if __name__ == "__main__":
    main()

```

(O arquivo `desenho.py` foi resumido acima por conter apenas dados — a lista completa dos 7 estágios da forca, sem lógica adicional — e está no repositório na íntegra.)

12. Comparação

Critério	Versão inicial	Versão refatorada
Funcionamento correto	3	5
Clareza	2	5
Organização	1	5
Legibilidade	2	5
Tratamento de erros	1	5
Facilidade de manutenção	1	5

A versão inicial funciona para o caminho feliz (palavras sem hífen, entradas sempre válidas), por isso recebe nota 3 e não 1 em “funcionamento correto” — mas falha em casos de borda que a versão refatorada trata explicitamente.

13. Reflexão

Onde a IA mais ajudou? Na padronização de boas práticas (uso de `set` em vez de lista com checagem manual, `dataclass` para agrupar estado, separação em módulos) e na geração dos testes automatizados, incluindo casos de borda que eu não teria pensado em cobrir sozinha de imediato (entrada vazia, letra repetida).

Onde a IA errou? A primeira versão refatorada entregue pela IA ainda considerava que *toda* a palavra precisava ser “adivinhada” letra a letra — não previu o caso de uma palavra composta com hífen, que só apareceu quando eu testei manualmente jogando de verdade.

O que precisei modificar? A condição de vitória e a exibição da palavra, para tratar caracteres que não são letras (hífen, e por extensão espaço/apóstrofo) como já revelados, sem exigir que sejam “adivinhados”.

Consigo explicar o código sem consultar a IA? Sim — cada função de `forca.py` é pequena, pura e tem uma única responsabilidade (validar entrada, registrar tentativa, montar a palavra revelada, checar vitória/derrota), o que tornou fácil entender e depois explicar o que cada trecho faz.

Qual foi o principal risco identificado no uso da IA durante a atividade? Confiar demais nos testes automatizados como prova de correção. Os 14 testes iniciais passavam integralmente e mesmo assim havia um bug que tornava o jogo impossível de vencer com uma palavra específica — a IA (e eu) só enxergamos isso depois de um teste manual real, não pela suíte automatizada.

14. Take Away

“Programar com IA não significa deixar a IA programar por mim. Significa usar a IA para acelerar a parte mecânica do trabalho — sintaxe, boas práticas conhecidas, geração de testes — sem abrir mão de entender, testar e me responsabilizar por cada decisão que o código realmente toma.”

Cinco regras para um desenvolvedor utilizar IA de maneira responsável:

1. Nunca integrar uma sugestão da IA sem antes conseguir explicá-la com minhas próprias palavras.
2. Sempre testar manualmente, além dos testes automatizados — um bug pode existir exatamente no caso que ninguém pensou em testar.
3. Usar prompts estruturados (papel, contexto, restrições, formato) em vez de pedidos genéricos como “melhore esse código”.
4. Nunca compartilhar dados sensíveis (senhas, chaves de API, dados pessoais) em um prompt, mesmo que pareça mais rápido.
5. Tratar toda sugestão da IA como uma proposta a ser avaliada, nunca como uma resposta final — quem assina o commit é quem decide o que entra no código.

Anexo A — Autoavaliação segundo os critérios da atividade

Critério	Valor	Atendido
Definição adequada do problema	0,05	Sim — seção 1 a 4
Construção do prompt	0,10	Sim — seções 5 e 10
Desenvolvimento da automação	0,10	Sim — seções 6 e 11
Testes e validação	0,10	Sim — seção 8 (3 casos) + 16 testes automatizados
Refatoração e comparação	0,05	Sim — seções 11 e 12
Análise crítica do uso da IA	0,05	Sim — seções 7, 9 e 13
Organização do registro no GitHub	0,05	Sim — repositório com <code>antes/</code> , <code>depois/</code> , <code>tests/</code> , <code>README.md</code>
Total	0,50	

Anexo B — Checklist de entrega

- ☒ Defini o problema.
- ☒ Identifiquei entrada, processamento e saída.
- ☒ Registre o prompt utilizado.
- ☒ Registre a primeira versão do código.
- ☒ Analisei criticamente a solução.
- ☒ Executei pelo menos três testes (seção 8) além de 16 testes automatizados (`tests/test_forca.py`).
- ☒ Registre os resultados.
- ☒ Solicitei e analisei uma refatoração.
- ☒ Comparei as versões.
- ☒ Respondi à reflexão final.
- ☒ Organizei o arquivo no GitHub.

Referências

OLIVEIRA, Kadidja Valéria Reginaldo de. **Material do Estudante — Programação Assistida e Automação com Inteligência Artificial**. UDF Centro Universitário, Tendências em Ciência da Computação, Unidade II, 2026.

OLIVEIRA, Kadidja Valéria Reginaldo de. **Programação Assistida por Inteligência Artificial: o novo paradigma da colaboração Humano-Máquina**. Material de aula — Atividade prática. UDF Centro Universitário, 2026.

PYTHON SOFTWARE FOUNDATION. **The Python Standard Library**. Disponível em: <https://docs.python.org/3/library/>. Acesso em: 11 set. 2026.

ANTHROPIC. **Claude**. Disponível em: <https://claude.ai/>. Acesso em: 11 set. 2026.